

UVM Verification of a P4 Adder and a Windowed Register File

RTL and post-synthesis functional verification

Erfan Taheri

Verification scope

Registered P4 adder

- 32-bit sparse-tree carry-select datapath
- fixed two-cycle latency
- carry propagation and prefix-tree activation
- complete RTL coverage and gate-level reuse

Sequence-based stimulus, monitored transactions, independent prediction, and coverage-guided closure

Windowed register file

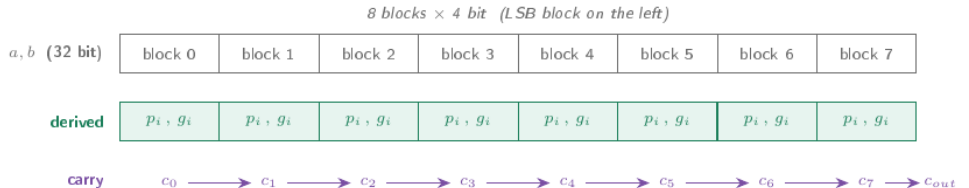
- eight overlapping windows and eight globals
- 136 physical 64-bit registers
- stateful address translation and boundary control
- cycle-accurate reference model and RAL globals

P4 verification goal and checklist

Goal Correct 33-bit arithmetic at the specified two-cycle latency, with direct evidence that the carry architecture was exercised.

- **Arithmetic**
 $\{C_{out}, S\} = A + B + C_{in}$
- **Timing** operands at T , result at $T + 2$
- **Reset** clear registers and flush invalid pipeline entries
- **Carry tree** P, G, carry-in, and ripple in all eight blocks
- **Critical cases** corners, full propagation, prefix terms
- **Reuse** same checks and coverage after synthesis

P4 adder RTL and verification focus



$p_i = \&(a_i \oplus b_i)$ (block fully propagates) $g_i = (a_i + b_i) \geq 16$ (block generates) $c_{i+1} = g_i \vee (p_i \wedge c_i)$
covered per block: $p_i, g_i, \text{ripple} = p_i \wedge c_i$ • crosses $\text{blk} \times p, \text{blk} \times g, \text{blk} \times \text{ripple}$ • illegal: $(p_i, g_i) = (1, 1)$

Pipeline

two-cycle alignment

Arithmetic

independent 33-bit result

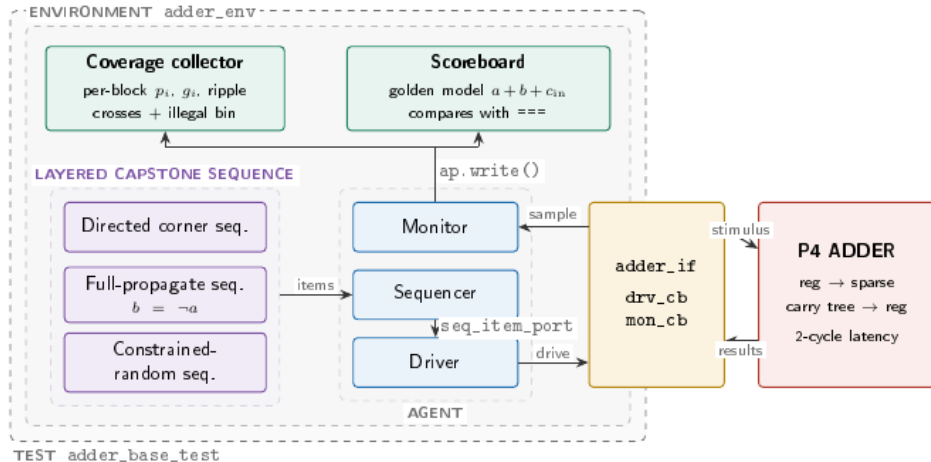
Carry tree

eight 4-bit blocks

Reset

registered wrapper

P4 UVM environment



P4 stimulus methodology

Selected stimulus: full-width carry propagation

```
add_prop_seq.svh · selected excerpt

task body();
  repeat (n) begin
    adder_seq_item req = adder_seq_item::type_id::create("req");

    start_item(req);
    assert(req.randomize() with { (a ^ b) == '1; });
    finish_item(req);
  end
endtask

// a ^ b == '1 makes every bit propagate an incoming carry.
```

- directed arithmetic corners
- constrained-random operands
- full propagation with $B = \neg A$
- non-masking prefix-tree vectors
- reset sequence for final closure
- pipeline drain before test completion

P4 functional coverage groups

Selected coverage model: each carry block must propagate, generate, and ripple

```
*** adder_coverage.svh - selected excerpt

covergroup cg_block with function sample
  (int id, bit p, bit g, bit carry_in);

  cp_block: coverpoint id { bins blk[] = {[0:NB-1]}; }
  cp_p:      coverpoint p;
  cp_g:      coverpoint g;
  cp_ripple: coverpoint (p & carry_in) {
    bins yes = {1}; bins no = {0};
  }

  x_block_p:  cross cp_block, cp_p;
  x_block_g:  cross cp_block, cp_g;
  x_block_ripple: cross cp_block, cp_ripple;

  impossible_pg: cross cp_p, cp_g {
    illegal_bins both = binsof(cp_p) intersect {1}
                      && binsof(cp_g) intersect {1};
  }
endgroup
```

Per 4-bit block index, propagate,
generate, carry ripple

Crosses and top level block by
condition, carry input/output, operand
corners, full propagation

Illegal state: $(P, G) = (1, 1)$

P4 coverage closure and results

Metric	Result
Scoreboard	601 checks, 0 mismatches
Functional coverage	100% (117/117)
Statement coverage	100%
Branch coverage	100%
Expression coverage	100%
Toggle coverage	100%
Assertion coverage	100% (14/14)
Filtered RTL coverage	100%
Post-synth. functional	100%

Closure actions

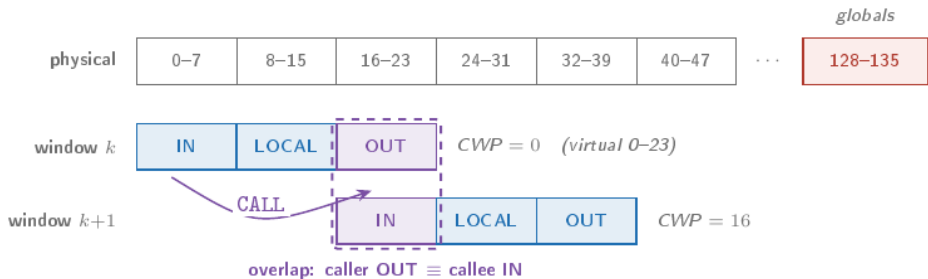
- reset sequence closed the final RTL bins
- targeted vectors activated masked prefix terms
- unsigned range types corrected operand bins
- the same environment ran on the netlist

Register-file verification goal and checklist

Goal Correct logical register mapping and window-management state on every clock, including the spill and fill boundaries.

- reset array, pointers, counters, and flags
- IN, LOCAL, OUT, and GLOBAL address mapping
- caller OUT to callee IN overlap
- registered reads and read-before-write
- call/return priority and disabled-state hold
- CWP, SWP, CANSAVE, and CANRESTORE
- spill/fill timing and boundary semantics
- RAL globals and mapped-netlist reuse

Windowed register file RTL



32 logical

IN/LOCAL/OUT/GLOBAL

136 physical

128 windowed + 8 fixed

8 windows

24 registers each

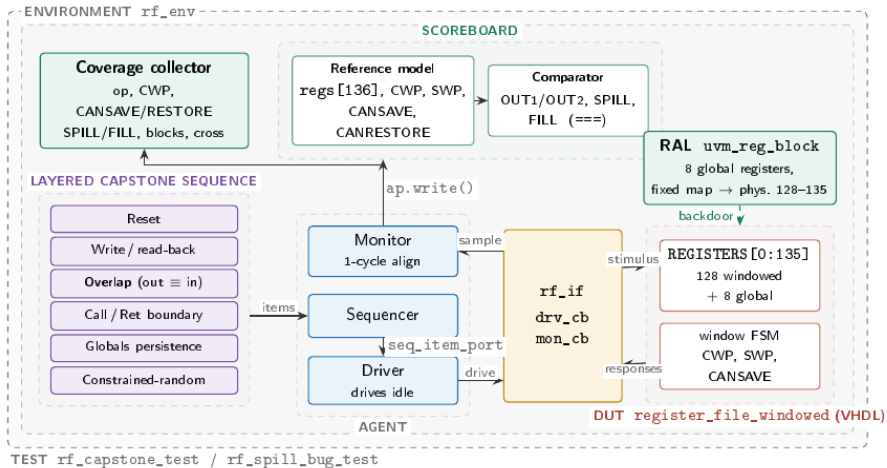
8 globals

fixed mapping

CALL/RET CWP/SWP SPILL/FILL

CANSAVE/CANRESTORE

Windowed register-file UVM environment



Cycle-accurate transaction and timing model

- **One transaction equals one clock** reset, call, return, access, or idle.
- **The driver applies an explicit idle** one-shot controls never remain asserted while the sequencer is empty.
- **The monitor uses a one-entry pipeline** inputs at cycle T are paired with registered outputs at $T + 1$.
- **The test drains the pipeline** the final response reaches the scoreboard before the objection drops.

Cycle-accurate reference model

State reproduced

- 136-entry physical register array
- CWP and SWP
- CANSAVE and CANRESTORE
- registered outputs and spill/fill pulses

Behavior reproduced

- logical-to-physical translation with wrap-around
- fixed mapping for global registers
- read-before-write ordering
- call/return priority and partial-reset behavior

The model advances once per monitored cycle and supplies CWP and counter state directly to coverage.

Register-file sequences and overlap check

Selected stimulus: caller OUT registers become callee IN registers

```
*** rf_seqs.svh - selected excerpt

class rf_overlap_seq extends rf_base_seq;
task body();
  for (int i = 0; i < N; i++)
    do_write(5'(2*N + i), 64'hA000_0000 + i); // caller OUT

    do_call(); // the physical storage now appears as callee IN

  for (int i = 0; i < N; i++)
    do_read(5'(i)); // callee IN
endtask
endclass
```

- write/read-back
- caller OUT to callee IN overlap
- globals across window changes
- repeated calls and returns to spill/fill
- constrained-random operations
- CWP sweep for coverage closure

Coverage of the internal FSM and counters

Window-state coverage and the ignored operation/CWP combinations

```
covergroup cg with function sample(
    int op_i, int cwp, int cansave, int canrestore,
    bit spill, bit fill, int blk_i);

cp_op:      coverpoint op_i { bins reset={0}; bins call={1};
                bins ret={2}; bins access={3}; bins idle={4}; }
cp_cwp:     coverpoint cwp    { bins w[] = {0,16,32,48,64,80,96,112}; }
cp_cansave: coverpoint cansave { bins v[] = {[0:F-1]}; }
cp_canrestore: coverpoint canrestore { bins v[] = {[0:F-1]}; }
cp_spill:   coverpoint spill;
cp_fill:    coverpoint fill;
cp_blk:     coverpoint blk_i { bins none={-1}; bins in={0}; bins loc={1};
                bins out={2}; bins glob={3}; }

x_op_cwp: cross cp_op, cp_cwp {
    ignore_bins reset_only_at_0 =
        binsof(cp_op) intersect {0} && !binsof(cp_cwp) intersect {0};
    ignore_bins call_never_wraps =
        binsof(cp_op) intersect {1} && binsof(cp_cwp) intersect {0};
    ignore_bins ret_never_at_top =
        binsof(cp_op) intersect {2} && binsof(cp_cwp) intersect {112};
}
endgroup
```

Seven reset/nonzero-CWP bins are structurally impossible. The CALL/CWP=0 and RET/CWP=112 exclusions instead reflect the implemented boundary behavior tied to the defect.

Coverpoints: operation, CWP, CANSAVE, CANRESTORE, SPILL, FILL, and addressed block.

Cross: every reachable operation at every reachable CWP position.

Functional coverage accounting

69/69

bins in the final covergroup

The source contains three ignore-bin declarations

- seven reset/nonzero-CWP bins are structurally impossible
- CALL at sampled CWP 0 follows the faulty boundary path
- RET at sampled CWP 112 follows the mirrored path

The saved reports show only the resulting 69/69-bin model. No preserved report supports a separate pre-exclusion percentage.

Only seven exclusions are inherently structural. Code coverage kept the two missed wrap assignments visible and exposed the defect.

RAL model and backdoor test

RAL model and the test that uses it

Fixed-map global registers

```
class rf_globals_block extends uvm_reg_block;
  rf_global_reg g[M];

  virtual function void build();
    default_map = create_map(
      "map", 0, DATA_W/8, UVM_LITTLE_ENDIAN);

    foreach (g[i]) begin
      g[i] = rf_global_reg::type_id::create(
        $sformatf("g%0d", i));
      g[i].configure(this);
      g[i].build();
      default_map.add_reg(
        g[i], i*(DATA_W/8), "RW");
      g[i].add_hdl_path_slice(
        $sformatf("REGISTERS[%0d]", NWIN+i),
        0, DATA_W);
    end
    lock_model();
  endfunction
endclass
```

rf_ral_test: backdoor use

```
function void build_phase(uvm_phase phase);
  env = rf_env::type_id::create("env", this);
  ral = rf_globals_block::type_id::create("ral");
  ral.build();
  ral.add_hdl_path("rf_top.dut_wrap.dut");
endfunction

task run_phase(uvm_phase phase);
  uvm_status_e status;
  uvm_reg_data_t rdata;
  bit [DATA_W-1:0] wdata;

  foreach (ral.g[i]) begin
    wdata = 64'hDEAD_0000 + i;
    ral.g[i].poke(status, wdata);
    ral.g[i].peek(status, rdata);
    if (rdata != wdata)
      'uvm_error("RAL", $sformatf(
        "g[%0d] mismatch", i))
  end
endtask
```

The RAL block covers only physical registers 128–135. The windowed region stays in the cycle-accurate reference model because its logical mapping changes with CWP.

Windowed register-file results

Metric	Result
Scoreboard	1,058 checks, 0 mismatches
Functional coverage	100% (69/69)
Statement coverage	94.73% (36/38)
Branch coverage	94.59% (35/37)
Toggle coverage	100% (438/438)
Assertion coverage	100%
Filtered overall RTL coverage	97.86%
Post-synthesis functional coverage	100% (69/69)
Timing	3.00 ns, slack 1.36 ns

Code coverage exposed the defect: the only misses were the two CWP wrap assignments at the spill/fill boundary.

Directed confirmation of the code-coverage finding

Expected window allocation



Observed RTL boundary path



1. Code coverage exposed it: the only RTL misses were the upward and downward CWP wrap assignments. Tracing them showed that spill/fill moves SWP while CWP and the allocation counters remain frozen.

2. Directed testing confirmed it: write a LOCAL register, exhaust the available windows, issue an overflowing CALL, then read the same logical LOCAL. The RTL returns the caller's value instead of the new window's reset value.

The missed wrap lines led to the frozen CWP/counter path; the specification-oriented LOCAL test then confirmed reuse of the caller's window.

Post-synthesis findings

Window-state reset

One mapped CANRESTORE bit could remain unknown after power-up under synchronous-only reset logic. The spill path worked while fill became unknown.

An asynchronous reset on the small control state removes the ambiguity.

Both issues were invisible in the RTL regression and appeared only at gate level.

Four-state monitoring

Two-state response fields converted a sampled x to zero before it reached the scoreboard.

DUT-sampled fields were changed to logic, preserving unknowns for strict comparison.

Summary and learning resources

P4 adder

Complete RTL coverage, 117/117 functional bins, independent arithmetic prediction, and unchanged gate-level reuse.

Windowed register file

Stateful reference model, internal FSM and counter coverage, 69/69 functional bins, RAL globals, and boundary-defect analysis.

Learning resources

- Ray Salemi, *UVM and SystemVerilog training course*, Siemens
- Ashok B. Mehta, *ASIC/SoC Functional Design Verification: A Comprehensive Guide to Technologies and Methodologies*